



LIONSDEV

Curso de Programação

MÓDULO 8

Banco de Dados com MongoDB

Professor Nicolas Cardoso Motta



Revisando o Módulo 7



HTTP

Método + rota na requisição; status na resposta: 200, 201, 400, 404...



Dados da requisição

`req.params` (na rota), `req.query` (depois do ?) e `req.body` (JSON).



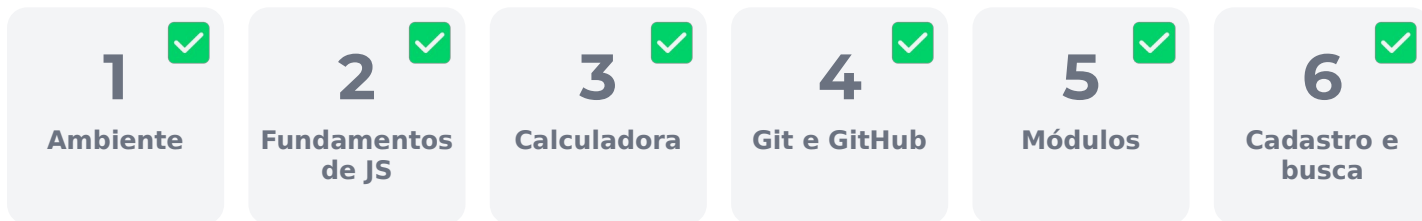
Middlewares

Funções no meio do caminho: validam, barram ou deixam passar com `next()`.

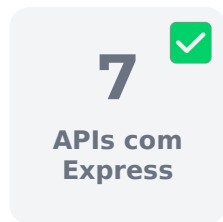
Pergunta para abrir: reinicie o servidor do Módulo 7. O que aconteceu com os flashcards cadastrados?

Mapa do curso

BASE: JAVASCRIPT, GIT E ORGANIZAÇÃO



BACKEND

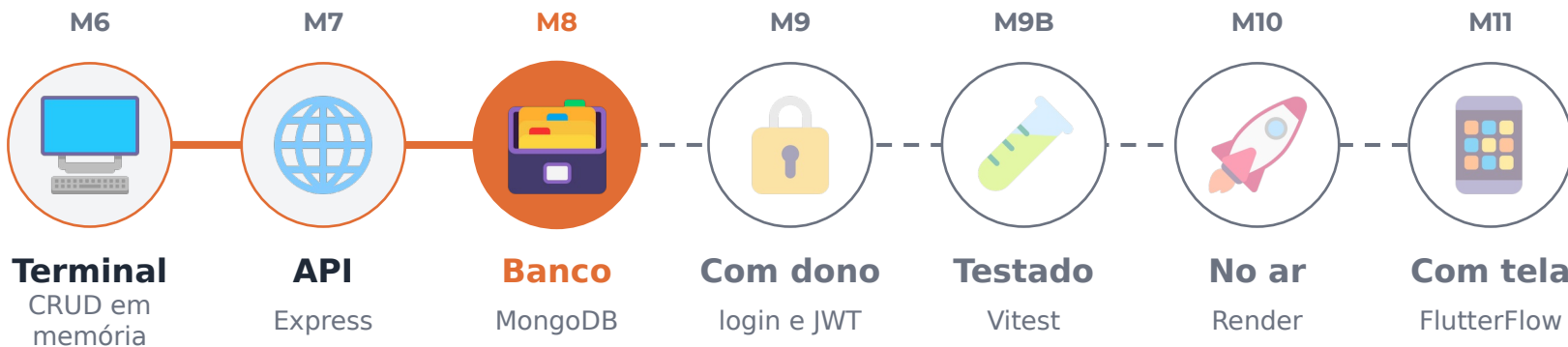


FRONTEND E PROJETO



Você está aqui: **Módulo 8**, Banco de Dados com MongoDB

O projeto flashcards



NO MÓDULO 8

Os baralhos e flashcards saem da memória e vão para o MongoDB: dois models do Mongoose, ligados por referência. Reiniciar o servidor não apaga mais nada.



Objetivo

- **Compreender o que é um banco de dados e por que usar um.**
- **Diferenciar bancos relacionais e não relacionais.**
- **Entender documentos, coleções e schema design no MongoDB.**
- **Configurar o MongoDB Atlas e proteger a conexão com variáveis de ambiente.**
- **Usar o Mongoose, com async/await e try/catch, para levar os flashcards para o banco.**
- **Atualizar só uma parte do documento e excluir sem perder os dados (soft delete).**

**Até agora, todos os dados viviam na
memória. Fechou o servidor, os
baralhos sumiram.**

Os limites do armazenamento em memória



Perda de dados

Reiniciou ou encerrou a aplicação,
as informações somem.



Difícil compartilhar

Cada servidor tem a sua própria
cópia dos dados na RAM.



Escalabilidade

A memória é limitada e cara para
guardar muita informação.



Consultas limitadas

Buscas e filtros ficam todos por
conta do nosso código.

Da memória para o banco

Antes: memória

```
// os dados vivem num array  
let baralhos = []  
  
baralhos.push(novo)
```

Agora: MongoDB

```
// os dados vivem no banco  
const baralhos = await Baralho.find()  
  
await Baralho.create(novo)
```

A lógica das rotas continua parecida. O que muda é **onde os dados ficam guardados**.



O que é um banco de dados?

Banco de dados



Armazena e organiza

Um sistema para guardar, organizar e gerenciar dados de forma estruturada.



Persistência

Os dados continuam lá mesmo depois que a aplicação é desligada.



Gestão centralizada e segura

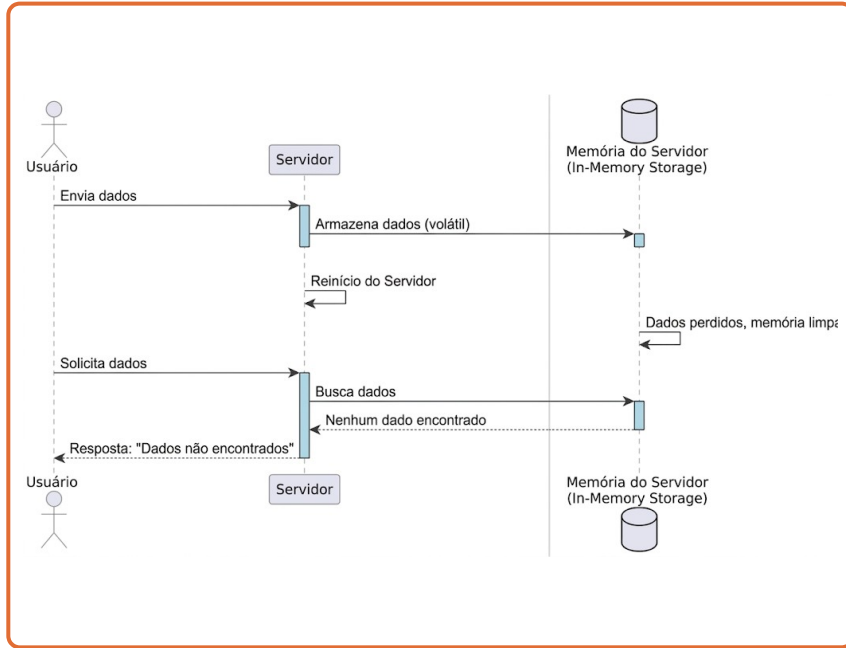
Um único lugar, com controle de acesso, que vários servidores podem usar.



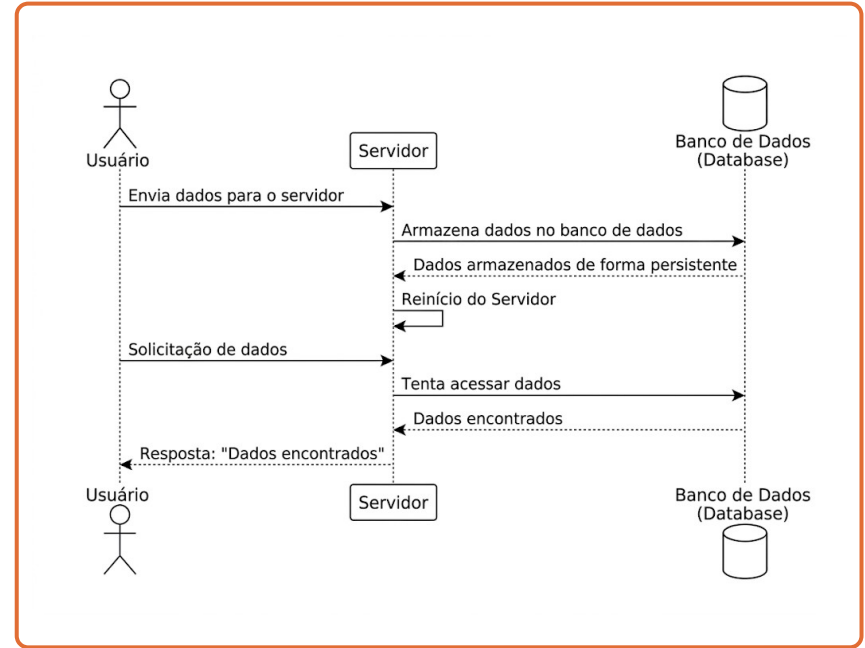
Consultas eficientes

Buscar, filtrar e ordenar milhões de registros com rapidez.

Memória x banco de dados



Memória



Banco de dados



Tipos de banco de dados

Relacional x não relacional



Relacional (SQL)

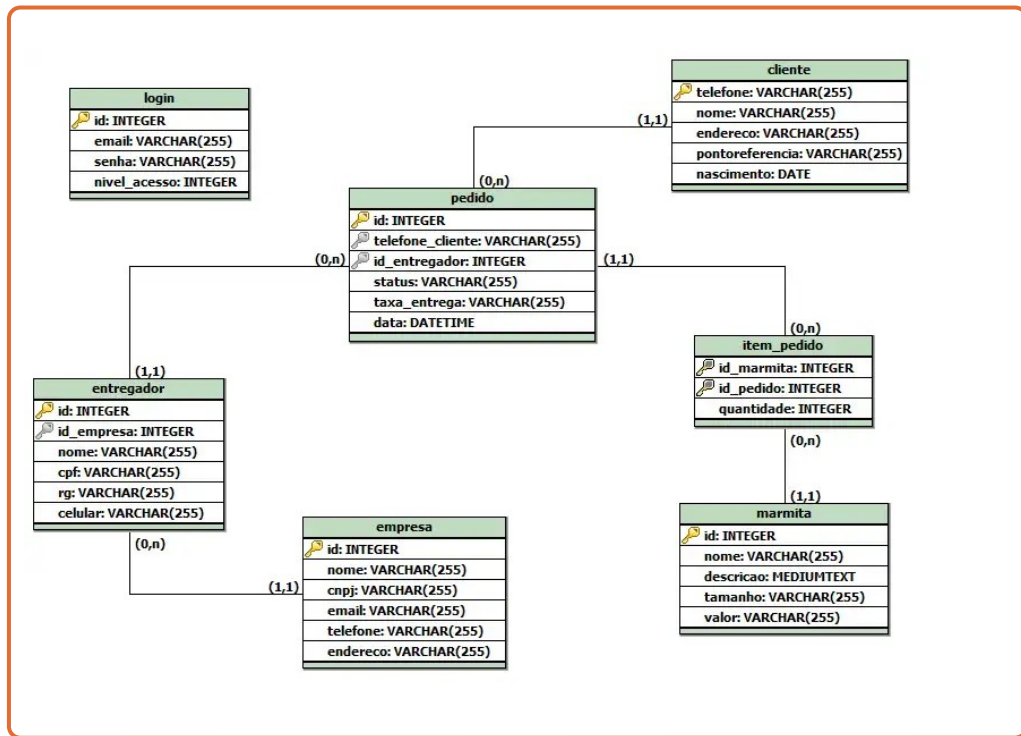
- Dados em tabelas, com linhas e colunas.
- Estrutura fixa, definida antes.
- Relações por chaves; consultas em SQL.



Não relacional (NoSQL)

- Dados em documentos parecidos com JSON.
- Estrutura flexível: cada documento pode variar.
- Documentos agrupados em coleções.

Como é um banco relacional



Tabelas

Linhas são os registros; colunas, as características.

Chaves

A primária identifica cada linha; a estrangeira liga uma tabela a outra.

Exemplos de bancos relacionais



MySQL

Popular, fácil de usar e muito presente na web.



PostgreSQL

Robusto, completo e de código aberto.



SQL Server

Da Microsoft, comum em empresas.

MongoDB

É o banco que vamos usar no curso.



Orientado a documentos

Guarda os dados num formato parecido com JSON: quase um objeto JavaScript.



Flexível

Não exige que todos os documentos tenham os mesmos campos.



Na nuvem, de graça

O MongoDB Atlas oferece um cluster gratuito para estudar.



O mesmo dado nos dois modelos

SQL

(Tabela e Colunas: Estrutura Rígida)

id	pergunta	resposta	id_baralho
1	O que é Node.js?	Ambiente de execução JavaScript	1

MongoDB

(Documento JSON: Estrutura Flexível)

```
{
  "_id": "507f1f77bcf86cd799439011",
  "pergunta": "O que é Node.js?",
  "resposta": "Ambiente de execução JavaScript",
  "idBaralho": "60d5ecfd0e0b7400155b9a88"
}
```

Tabela com colunas fixas x documento com chave e valor.

Por que MongoDB?



Feito para JavaScript

Os documentos parecem objetos JS: menos tradução entre o código e o banco.



Flexível

Dá para evoluir a estrutura dos dados sem migrações complicadas.



Escalável

Aguenta muito volume e muitas requisições.



Atlas gratuito

Um banco na nuvem, pronto em minutos.



Documentos e coleções

Estrutura de um documento

```
{
  "_id": ObjectId("5f4f7fef2d4b45b7f11b6d7a"),
  "nome": "Joao",
  "idade": 29,
  "profissao": "Programador"
}
```

documento → uma unidade de dados: um usuário, um produto, um flashcard.

chave: valor → cada informação tem um nome (chave) e um valor.

_id → criado automaticamente pelo MongoDB: um ObjectId único, não um número 1, 2, 3.

Banco, coleções e documentos

Banco de dados: flashcards_modulo08

Coleção: baralhos

```
{ titulo: "JavaScript" }
```

```
{ titulo: "Matemática" }
```

Coleção: flashcards

```
{ pergunta: "O que é  
let?" }
```

```
{ pergunta: "Quanto é 1+1?"  
}
```

Coleção: usuarios

```
{ nome: "Ana" }
```

```
{ nome: "Bia" }
```



Schema design

Como organizar os dados relacionados

Embedding x referência

Embedding: tudo junto

```
// coleção baralhos
{
  "titulo": "JS",
  "flashcards": [
    { "pergunta": "let?" },
    { "pergunta": "const?" }
  ]
}
```

Referência: coleções separadas

```
// coleção baralhos
{ "_id": "b1", "titulo": "JS" }

// coleção flashcards
{ "pergunta": "let?", "baralho": "b1" }
{ "pergunta": "const?", "baralho": "b1" }
```

Embedding quando os dados são sempre lidos juntos e são poucos. **Referência** quando crescem muito ou são usados sozinhos. Os flashcards usam referência.

Relacionamentos entre documentos

1 : 1

Um para um

Um usuário tem um perfil.

1 : N

Um para muitos

Um baralho tem vários flashcards.

N : N

Muitos para muitos

Alunos fazem várias turmas; turmas têm vários alunos.

É o mesmo relacionamento por IDs do Módulo 6. Agora o ID é o `_id` do MongoDB.



MongoDB Atlas

O nosso banco de dados na nuvem

Criando o banco no Atlas

- 1 Crie a conta**
Em `mongodb.com/atlas`, com e-mail, Google ou GitHub.
- 2 Escolha o plano gratuito**
O cluster M0 (Free) é suficiente para o curso.
- 3 Crie o cluster**
Um "servidor" de banco de dados só seu.
- 4 Crie um usuário do banco**
Nome e senha que a aplicação vai usar para se conectar. Não é o login do site!

Conectando a aplicação

- 5 **Método de conexão: Drivers**
Escolha a opção para conectar pelo código.
- 6 **Driver Node.js**
Vamos usar o Mongoose, que usa esse driver por baixo.
- 7 **Copie a connection string**
E guarde no arquivo `.env`, nunca direto no código.
- 8 **Libere o IP (Network Access)**
Adicione o IP da máquina. `0.0.0.0/0` libera qualquer IP: só para estudo.

Passo a passo no Atlas

Deploy your cluster

Use a template below or set up advanced configuration options. You can also edit these configuration options once the cluster is created.

☐ M10

\$0.12/hour

Dedicated cluster for development environments and low-traffic applications.

STORAGE	RAM	vCPU
10 GB	2 GB	2 vCPUs

☐ Flex

From \$0.011/hour
Up to \$30/month

For development and testing, with on-demand burst capacity for unpredictable traffic.

STORAGE	RAM	vCPU
5 GB	Shared	Shared

☒ Free

For learning and exploring MongoDB in a cloud environment.

STORAGE	RAM	vCPU
612 MB	Shared	Shared

Free forever! Your free cluster is ideal for experimenting in a limited sandbox. You can upgrade to a production cluster anytime.

Plano gratuito

Configurations

Name
You cannot change the name once the cluster is created.

ClusterName

Provider

☒ AWS ☐ Google Cloud ☐ Azure

Region

☒ Sao Paulo (sa-east-1) ☐ Low carbon emissions

Tag (optional)

Create your first tag to categorize and label your resources; more tags can be added later. [Learn more.](#)

Select an access key Select an access role

I'll do this later

Go to Advanced Configuration

Create Deployment

Criação do cluster

Connect to ClusterTeste

1 Set up connection security 2 Choose a connection method 3 Connect

You need to secure your MongoDB Atlas cluster before you can use it. Set which users and IP addresses can access your cluster now. [Read more.](#)

1. Add a connection IP address

Your current IP address () has been added to enable local connectivity. Only an IP address you add to your Access List will be able to connect to your project's cluster. Add more later in [network access](#).

2. Create a database user

This first user will have [atlasAdmin](#) permissions for this project.

We autogenerated a username and password. You can use this or create your own.

3 You'll need your database user's credentials in the next step. Copy the database user password.

Username Password

test_db_user

Create Database User

Close

Choose a connection method

Usuário do banco

3. Add your connection string into your application code

Use this connection string in your application

☒ SRV Connection String ☐ Show Password

☐ View full code sample

mongodb+srv://test_db_user:<db_password>@cluster-teste.baqzhz.mongodb.net/?appName=ClusterTeste

Replace <db_password> with the password for the test_db_user database user. Ensure any option params are URL encoded.

RESOURCES

[Get started with the Node.js Driver](#)

[Access your Database Users](#)

[Node.js Starter Sample App](#)

[Troubleshoot Connections](#)

Go Back

Done

Connection string



Cuidado: a senha na connection string

Caracteres como @, : ou / na senha **quebram a connection string.**

- A URI usa esses caracteres como separadores: uma senha com @ faz o Mongoose achar que o endereço começou antes.
- Crie uma senha só com letras e números, ou codifique os especiais (@ vira %40).
- Erro de autenticação? Confira o usuário do banco (passo 4), que não é o login do site.

```
mongodb+srv://aluno:<senha>@cluster0.xxxxx.mongodb.net/flashcards
```



Variáveis de ambiente

O arquivo .env

```
# .env
MONGO_URI=mongodb+srv://aluno:senha@cluster0.xxxxx.mongodb.net/flashcards
PORT=3000
```

A connection string tem usuário, senha e endereço do banco: informação sensível. Por isso ela fica no .env, fora do código.

Lendo o .env com o dotenv

```
import dotenv from 'dotenv'  
dotenv.config()  
  
console.log(process.env.MONGO_URI)  
console.log(process.env.PORT)
```

dotenv → pacote que lê o arquivo **.env**: **npm install dotenv**.

config() → carrega as variáveis. Chame antes de usar qualquer uma.

process.env → o objeto onde o Node guarda as variáveis de ambiente.



Cuidado: o .env não vai para o GitHub

Um **.env** commitado expõe a senha do banco para **qualquer pessoa**.

- Coloque o **.env** no **.gitignore** antes do primeiro commit.
- Suba um **.env.example**, com os nomes das variáveis e valores de exemplo, para quem clonar saber o que configurar.
- Vazou? Troque a senha do usuário do banco no Atlas na hora.

```
# .gitignore
node_modules/
.env
```



Mongoose

Como o Mongoose funciona?

O Mongoose é a biblioteca que conecta a aplicação ao MongoDB:



Com ele, definimos um **Schema** (o formato dos dados) e um **Model** (a interface para o CRUD).

Estrutura de pastas

```
projeto/
├── src/
│   ├── server.js      (rotas)
│   ├── database.js    (conexão)
│   └── models/
│       ├── Baralho.js
│       └── Flashcard.js
├── .env
├── .env.example
└── package.json
```

database.js → conecta ao MongoDB.

models/ → um arquivo por coleção, com o Schema e o Model.

instalação → `npm install express mongoose dotenv`

Conectando: database.js

```
import mongoose from 'mongoose'

async function conectarBanco() {
  try {
    await mongoose.connect(process.env.MONGO_URI)
    console.log('Conectado ao MongoDB')
  } catch (error) {
    console.error('Erro ao conectar:', error.message)
    process.exit(1)
  }
}

export default conectarBanco
```

Se a conexão falhar, não adianta a API subir: o `process.exit(1)` encerra o programa com uma mensagem clara.

O Schema e o Model: Baralho.js

```
const baralhoSchema = new mongoose.Schema(  
  {  
    titulo: {  
      type: String,  
      required: [true, 'O título é obrigatório.'],  
      trim: true,  
    },  
    descricao: { type: String, default: '' },  
  },  
  { timestamps: true }  
)  
  
const Baralho = mongoose.model('Baralho', baralhoSchema)  
export default Baralho
```

required → campo obrigatório, com a mensagem de erro.

trim → tira os espaços das pontas antes de salvar.

timestamps → cria **createdAt** e **updatedAt** sozinho.

model('Baralho') → vira a coleção **baralhos**: minúscula e no plural.

Referenciando outro model: Flashcard.js

```
const flashcardSchema = new mongoose.Schema(  
  {  
    pergunta: { type: String, required: true, trim: true },  
    resposta: { type: String, required: true, trim: true },  
    baralho: {  
      type: mongoose.Schema.Types.ObjectId,  
      ref: 'Baralho',  
      required: true,  
    },  
  },  
  { timestamps: true }  
)
```

O campo baralho guarda o `_id` do baralho, e o `ref` diz para qual model esse id aponta: o relacionamento 1:N.

As operações do Mongoose

`create()`

Cria um documento.

antes: push

`find()`

Lista (com filtro opcional).

antes: o array / filter

`findById()`

Busca pelo `_id`.

antes: find

`findByIdAndUpdate()`

Atualiza pelo `_id`.

antes: `findIndex` + atribuição

`findByIdAndDelete()`

Remove pelo `_id`.

antes: `findIndex` + splice

`deleteMany()`

Remove vários de uma vez.

antes: laço com splice

populate(): trazendo o documento relacionado

```
const flashcards = await Flashcard.find({ baralho: id })  
// [{ pergunta: '...', baralho: '665f1c...' }]  
  
const completos = await Flashcard.find({ baralho: id }).populate('baralho', 'titulo')  
// [{ pergunta: '...', baralho: { _id: '665f1c...', titulo: 'JavaScript' } }]
```

O populate usa o ref do Schema para trocar o id pelo documento. É o "garçom" do Módulo 6, feito pelo Mongoose.

Ordenar, limitar e contar

```
await Baralho.find().sort({ createdAt: -1 }) // mais novos primeiro
await Baralho.find().limit(10) // só 10
await Baralho.find({ titulo: /java/i }) // busca parcial
await Baralho.countDocuments() // quantos existem
```

Buscas e filtros que antes eram laços no nosso código agora são feitos pelo banco: bem mais rápido com muitos dados.



Cuidado: unique não é validação

O **unique: true** cria um índice no banco. E-mail repetido não dá erro de validação: dá **E11000 duplicate key**.

- Trate esse erro pelo código **11000** e responda **409 Conflict**.
- Use **lowercase** e **trim** no campo: "Ana@email.com " e "ana@email.com" viram o mesmo e-mail.

```
email: { type: String, required: true, unique: true, lowercase: true, trim: true }  
  
if (error.code === 11000) return res.status(409).json({ message: 'E-mail já cadastrado' })
```



Operações assíncronas

O banco está longe: as respostas demoram

Promise: uma resposta que ainda vai chegar

Enquanto o MongoDB responde, o Node.js continua atendendo outras requisições. Toda operação do Mongoose devolve uma Promise.



Pending

A operação ainda está acontecendo.



Fulfilled

Terminou com sucesso e trouxe o resultado.



Rejected

Deu erro no caminho.

.then() x async/await

.then() e .catch()

```
Baralho.find()
  .then((baralhos) => {
    console.log(baralhos)
  })
  .catch((erro) => {
    console.log(erro.message)
  })
```

async / await

```
async function listar() {
  try {
    const baralhos = await Baralho.find()
    console.log(baralhos)
  } catch (erro) {
    console.log(erro.message)
  }
}
```

O async/await não elimina a Promise: ela continua lá. O código só fica parecido com uma sequência normal, fácil de ler.

Simulando uma espera

```
function esperar(ms) {  
  return new Promise((resolve) => setTimeout(resolve, ms))  
}  
  
async function main() {  
  console.log('Antes da espera')  
  await esperar(2000)  
  console.log('Depois de 2 segundos')  
}  
  
main()
```

Saída no terminal

```
$ node async.js  
Antes da espera  
Depois de 2 segundos
```

O `await` pausa só esta função até a `Promise` terminar. E ele só funciona dentro de uma função `async`.



Sem o await: o que aparece?

```
app.get('/baralhos', async (req, res) => {  
  const baralhos = Baralho.find() // esqueceu o await  
  res.json(baralhos)  
})
```

Pense antes de rodar: qual é a saída?



Cuidado: esqueceu o await

A variável recebe a **consulta**, não os dados. O **res.json** não consegue converter a consulta em JSON, e a rota responde **500**.

- Sem **await**, **Baralho.find()** devolve uma consulta que ainda nem rodou.
- Toda chamada ao Mongoose precisa de **await**, e a função precisa ser **async**.

```
const baralhos = await Baralho.find() // agora sim: a lista de baralhos
```

try/catch com async/await

```
app.get('/baralhos', async (req, res) => {  
  try {  
    const baralhos = await Baralho.find()  
    return res.status(200).json(baralhos)  
  } catch (error) {  
    return res.status(500).json({  
      message: 'Erro ao listar',  
      error: error.message,  
    })  
  }  
})
```

Se o banco falhar, a execução pula para o catch: o cliente recebe um **500** com a mensagem, e o servidor não cai. É o try/catch do Módulo 2 em ação.



Flashcards com MongoDB

CREATE: antes e agora

Antes: memória

```
const titulo = req.body.titulo
const resultado = adicionarBaralho(titulo)
// push no array
```

Agora: MongoDB

```
const baralho = await Baralho.create(req.body)
// o required do Schema já valida
```

A validação do título saiu do nosso código e foi para o Schema.

POST /baralhos

```
app.post('/baralhos', async (req, res) => {  
  try {  
    const baralho = await Baralho.create(req.body)  
    return res.status(201).json({ message: 'Criado!', baralho })  
  } catch (error) {  
    return res.status(400).json({ message: error.message })  
  }  
})
```

Aqui o erro vira 400: quase sempre é validação do Schema, como o título faltando.

Testando: POST /baralhos

Requisição

POST /baralhos

```
{  
  "titulo": "MongoDB"  
}
```

Resposta

201 Created

```
{  
  "message": "Baralho criado com sucesso",  
  "baralho": {  
    "_id": "665f1c...",  
    "titulo": "MongoDB",  
    "descricao": "",  
    "createdAt": "..."  
  }  
}
```

Confira no Atlas (Browse Collections): o documento está lá, na coleção baralhos.

GET /baralhos/:id

```
app.get('/baralhos/:id', async (req, res) => {  
  try {  
    const baralho = await Baralho.findById(req.params.id)  
    if (!baralho) {  
      return res.status(404).json({ message: 'Não encontrado' })  
    }  
    return res.status(200).json(baralho)  
  } catch (error) {  
    return res.status(500).json({ message: error.message })  
  }  
})
```

Se o id não existe, o findById devolve null: é o nosso **404**. O GET /baralhos é igual, com Baralho.find().



Cuidado: id que não é um ObjectId

Um id com formato errado, como `/baralhos/123`, não devolve null: lança um **CastError**.

- Sem tratar, cai no catch e vira um 500, como se o servidor tivesse quebrado.
- Valide antes com `mongoose.isValidObjectId(id)` e responda 400.

```
if (!mongoose.isValidObjectId(req.params.id)) {  
  return res.status(400).json({ message: 'Id inválido' })  
}
```

PUT /baralhos/:id

```
const atualizado = await Baralho.findByIdAndUpdate(
  req.params.id,
  req.body,
  { new: true, runValidators: true }
)

if (!atualizado) {
  return res.status(404).json({ message: 'Baralho não encontrado' })
}
return res.status(200).json({ baralho: atualizado })
```

O new: true devolve o documento depois da atualização. O runValidators faz o update respeitar as validações do Schema, que por padrão ele ignora.

DELETE /baralhos/:id

```
const deletado = await Baralho.findByIdAndDelete(req.params.id)

if (!deletado) {
  return res.status(404).json({ message: 'Baralho não encontrado' })
}

// cascata manual: apaga os flashcards desse baralho
await Flashcard.deleteMany({ baralho: req.params.id })

return res.status(200).json({ message: 'Baralho e flashcards deletados' })
```

O MongoDB não apaga os flashcards sozinho: o deleteMany faz a "cascata" que o laço de trás para frente fazia no Módulo 7.

Todas as rotas do sistema

Baralhos	Flashcards
POST /baralhos	POST /flashcards
GET /baralhos	GET /flashcards
GET /baralhos/:id	GET /flashcards/:id
PUT /baralhos/:id	PUT /flashcards/:id
DELETE /baralhos/:id	DELETE /flashcards/:id
GET /baralhos/:id/flashcards	

Agora as rotas usam o plural e o id é o `_id` do MongoDB.



Atualização parcial e soft delete

Como APIs de verdade atualizam e removem dados

O que o findByIdAndUpdate envia ao banco

O cliente mandou só a pergunta nova. O que acontece com a resposta e o baralho?

```
await Flashcard.findByIdAndUpdate(id, { pergunta: 'Nova?' })
```

```
// o Mongoose envia para o MongoDB:
```

```
// { $set: { pergunta: 'Nova?', updatedAt: ... } }
```

O \$set altera **só** os campos enviados: o resto fica como estava. Ou seja, o nosso PUT já funciona como um **PATCH**.



Cuidado: o req.body inteiro no update

Com `findByIdAndUpdate(id, req.body)`, o cliente altera **qualquer** campo: até trocar o flashcard de baralho.

- Numa API com login (Módulo 9), seria trocar o dono de um registro.
- Monte o update só com os campos permitidos: o que não está na lista é ignorado.

```
const permitidos = ['pergunta', 'resposta']
const dados = {}
for (const campo of permitidos) {
  if (req.body[campo] !== undefined) dados[campo] = req.body[campo]
}
await Flashcard.findByIdAndUpdate(req.params.id, dados, { new: true, runValidators: true })
```

Apagando um campo: \$unset

O flashcard ganhou um campo opcional, a **dica**. Como o cliente pede para apagar a dica?

```
// body: { "dica": null } → apagar a dica
const set = {}
const unset = {}
for (const campo of ['pergunta', 'resposta', 'dica']) {
  const valor = req.body[campo]
  if (valor === null) unset[campo] = 1 // null: apagar
  else if (valor !== undefined) set[campo] = valor // veio: alterar
}

const atualizado = await Flashcard.findByIdAndUpdate(
  req.params.id,
  { $set: set, $unset: unset },
  { new: true, runValidators: true }
)
```

Campo que não veio (undefined) fica como está. E o runValidators barra apagar a pergunta, que é obrigatória: ValidationError, resposta 400.

Hard delete x soft delete



Hard delete

- O documento é apagado do banco: `findByIdAndDelete`.
- Simples, mas não tem volta.



Soft delete

- O documento fica, marcado com a data da exclusão: `excluidoEm`.
- Dá para desfazer, manter o histórico e não quebrar relacionamentos.

Soft delete não serve para tudo: dados pessoais que o usuário pediu para apagar precisam sair de verdade (LGPD).

Soft delete: o Schema e a rota DELETE

```
// no Schema
excluidoEm: { type: Date, default: null },

// DELETE /flashcards/:id
const excluido = await Flashcard.findOneAndUpdate(
  { _id: req.params.id, excluidoEm: null },
  { excluidoEm: new Date() },
  { new: true }
)
if (!excluido) {
  return res.status(404).json({ message: 'Flashcard não encontrado' })
}
return res.status(204).send()
```

**O filtro excluidoEm: null impede excluir de novo o que já foi excluído: vira 404.
E o 204 do Módulo 7 volta aqui.**

Escondendo os excluídos

```
// listar: só os que não foram excluídos
await Flashcard.find({ excluidoEm: null })

// buscar um: o _id e não excluído
await Flashcard.findOne({ _id: req.params.id, excluidoEm: null })

// restaurar: PATCH /flashcards/:id/restaurar
await Flashcard.findOneAndUpdate(
  { _id: req.params.id, excluidoEm: { $ne: null } },
  { excluidoEm: null },
  { new: true }
)
```

Buscar por excluidoEm: null também encontra os documentos antigos, que nem têm o campo. O \$ne significa "diferente de".



Cuidado: o findById não sabe do soft delete

Um `findById` devolve o flashcard **mesmo excluído**. O soft delete só funciona se **toda** consulta filtrar.

- Troque `findById(id)` por `findOne({ _id: id, excluidoEm: null })`.
- O mesmo vale para listar, atualizar e contar: qualquer consulta sem o filtro "ressuscita" os excluídos.
- No Módulo 9, as consultas ficam num lugar só, o repository: é lá que o filtro entra, uma vez.



Hora da Prática

Atividade Prática

MongoDB

1. Crie a pasta `modulo08`, o cluster no Atlas e o arquivo `.env` com a `connection string`.
2. Use o flashcards dos slides (ou do repositório) para entender o Mongoose.
3. Leve as APIs do Módulo 7 para o MongoDB: um Schema e um Model por recurso.
4. Troque as operações em memória pelas do Mongoose, com `async/await` e `try/catch`.
5. Teste no Postman e confira os documentos no Atlas.



Dica: coloque o `.env` no `.gitignore` antes do primeiro commit e suba um `.env.example`.



Desafio: troque o `DELETE` de um recurso por `soft delete` e crie a rota de restaurar.

Lista de exercícios



Lista de código

Schema, model, create, find e findById: complete, ache o bug e preveja o resultado.



APIs com banco

Academia, Biblioteca, Cantina, Imóvel e Petshop Lions: uma API completa em cada.



Desafio

Um CRUD com Mongoose escrito do zero.

No repositório: [modulo08/lista_de_exercicios/](#)

Material do módulo



Aponte a câmera do celular

`github.com/nicolassmotta/curso-lions-dev/
tree/main/modulo08`



conteudo/
teoria comentada e o cheat sheet do módulo



exercicios_slides/
os exercícios feitos em aula



lista_de_exercicios/
listas para praticar em casa



exercicios_resolvidos/
gabaritos, para conferir depois de tentar



Fim!

No Módulo 9: senhas, login com JWT e o padrão MVC.